

REVIEW

Open Access



Seamless transitions: a comprehensive review of live migration technologies

Sima Attar-Khorasani^{1,2*}, Lincoln Sherpa^{1,2}, Matthias Lieber^{1,2} and Siavash Ghiasvand^{2,3}

Abstract

Live migration, a technology enabling seamless transition of operational computational entities between various hosts while preserving continuous functionality and client connectivity, has been the subject of extensive research. However, existing reviews often overlook critical technical aspects and practical challenges integral to the usage of live migration techniques in real-world scenarios. This work bridges this gap by integrating the aspects explored in existing reviews together with a comprehensive analysis of live migration technologies across multiple dimensions, with focus on migration techniques, migration units, and infrastructure characteristics. Despite efforts to make live migration widely accessible, its reliance on multiple system factors can create challenges. In certain cases, the complexities and resource demands outweigh the benefits, making its implementation hard to justify. The focus of this work is mainly on container-based and virtual machine-based migration technologies, examining the current state of the art and the disparity in adoption between these two approaches. Furthermore, this work explores the impact of migration objectives and operational constraints on the usability and efficacy of existing technologies. By outlining current technical challenges and providing guidelines for future research and development directions, this work serves a dual purpose: first, to equip enthusiasts with a valuable resource on live migration, and second, to contribute to the advancement of live migration technologies and their practical implementation across diverse computing environments.

Keywords Live migration, Container, Virtual machine, Live migration survey, Checkpoint/Restore tools

Introduction

Live migration denotes seamless transfer of a running computational entity, such as a Virtual Machine (VM), between various hosts while the continuous operation and client connectivity is maintained. First demonstrated in 2005 Clark et al. [1], live migration has evolved to

encompass applications in Edge, cloud, and (HPC), offering various benefits in flexibility, scalability, fault tolerance, and resource efficiency.

Various studies reviewed live migration. However, most reviews focus primarily on categorizing existing techniques and comparing their performance metrics, without delving into the fundamental technical aspects of live migration. Others either overlook the practical challenges in performing live migration, or they focus heavily on particular applications or environments. Therefore, a significant gap exists in studying live migration technologies and real-world operational scenarios constrained by practical considerations. This work addresses this gap by analyzing existing technologies, exploring unique challenges, and examining migration specifications tailored

*Correspondence:

Sima Attar-Khorasani

sima.attar_khorasani@tu-dresden.de

¹Information Services and High Performance Computing (ZIH), TUD Dresden University of Technology, Dresden, Germany

²Center for Interdisciplinary Digital Sciences (CIDS), TUD Dresden University of Technology, Dresden, Germany

³Center for Scalable Data Analytics and Artificial Intelligence, (ScaDS.AI), Dresden/Leipzig, Germany

to diverse practical perspectives, focusing on four key dimensions namely migration objective, migration technique, migration unit, and infrastructure characteristics. The various perspectives and decision criteria are further summarized in a table structure to make the complexities and relations of live migration technologies better understandable, providing insights into how live migration solutions can be adapted to meet the distinct needs in various scenarios.

Among the four key dimensions in this work, the *migration objective* explores the drivers of migration and argues that the distinction between “live” and “cold” migration is determined by the specific downtime tolerance of the application. The *migration technique* encompasses various methods, such as pre-copy, post-copy, and Checkpoint/Restore (C/R) tools. The *migration unit* investigates the differences between containers and VMs, each with distinct requirements and technologies in the migration context, followed by an exploration of Hybrid and Emerging Technologies. The *infrastructure characteristics* explores how factors such as network conditions and environment influence migration strategies.

It is also crucial to note that underlying technologies, such as checkpointing, play a vital role in the live migration process. Such variations significantly influence the efficiency and effectiveness of live migration across different contexts.

This work provides researchers with a clear understanding of the current state-of-the-art in live migration, highlighting areas where current technologies excel and identifying gaps requiring further innovation. This work is organized as follows: Sect. 2 establishes the objectives of live migration. Section 3 presents a comprehensive overview of relevant studies and defines the scope of this research. Sections 4, 5, and 6 investigate migration techniques, units, and infrastructure characteristics, respectively. To conclude, Sect. 7 provides practical guidelines for live migration implementation.

Migration objectives

Live migration is not merely a technical capability but a strategic operation driven by specific infrastructure and application goals. The primary objectives for initiating migration generally fall into four distinct categories: resource efficiency, reliability and fault tolerance, network optimization, and security.

First, migration is the cornerstone of server consolidation, a strategy aimed at energy efficiency where virtual machines are packed onto the minimum number of physical hosts. This allows idle machines to be powered off or put into sleep mode, significantly reducing power consumption and operational costs Kaur and Chana [2], Imran et al. [3]. Second, it enables dynamic load balancing to mitigate performance variability. By continuously

monitoring resource thresholds, systems can detect “hotspots” (overloaded nodes) and “cold spots” (underutilized nodes) and proactively redistribute workloads. This ensures that (SLA)s are met by preventing resource bottlenecks before they degrade application performance Kaur and Chana [2], Lu et al. [4].

Critically, the capability to migrate workloads has profound implications for modern datacenter design. Historically, energy efficiency efforts focused primarily on physical infrastructure, such as utilizing Energy Star equipment, or strategically locating facilities (e.g., Google’s data centers on the Columbia River) to aid cooling Soma and Rukmini [5]. While efficient hardware like microblades has contributed to these goals, physical optimizations alone are insufficient to address the varying demands of cloud users Soma and Rukmini [5]. Live migration shifts the paradigm from hardware-centric to software-defined energy optimization. By decoupling the workload from the physical hardware, administrators can optimize energy consumption dynamically at the software level, compensating for hardware inefficiencies and reducing the need for massive static over-provisioning of resources Soma and Rukmini [5].

Aside from that, migration ensures proactive fault tolerance. Unlike reactive measures taken after a crash, proactive migration allows administrators to evacuate workloads from nodes exhibiting early signs of hardware degradation, such as correctable Error Correction Code (ECC) memory errors, to healthy nodes before a catastrophic failure occurs Sindi and Williams [6]. Additionally, modern strategies increasingly focus on network and bandwidth awareness. In distributed environments like Cloudlets or Edge computing, migration is used to minimize network traffic costs and reduce latency by moving services physically closer to the mobile end-user Imran et al. [3]. Finally, security has emerged as a critical objective. Migration enables the survival of services during cyber threats, allowing systems to move workloads away from nodes under Denial-of-Service (DoS) attacks or to trusted execution environments to maintain data integrity Imran et al. [3].

However, the success of the migration is strictly defined by the specific sensitivity of the application being migrated. There is no universal standard for “liveness”; rather, the requirements vary drastically across, for example, HPC, Big Data, (ML), and (IoT) scenarios. For example, in tightly-coupled HPC applications using Message Passing Interface (MPI), a downtime exceeding the MPI timeout (typically microseconds to milliseconds) can disrupt synchronization and crash the job, necessitating near-zero downtime migration solutions, Sindi and Williams [6], Garg et al. [7]. In contrast, Big Data and ML training scenarios, which are resource-intensive and long-running, often prioritize execution

Table 1 (continued)

Migration Issues			Migration Units			Env.	Migration Technology		C/R Tools	Container Engines		Virtualization Tools												
Mechanism	Application	Management	VM	Container	Network	Storage	Cloud/Cluster	Edge/Fog	Pre-Copy	Post-Copy	Hybrid Copy	CRIU	DMTCP	Others ¹	Docker	LXC	OpenVZ	Apptainer	Podman	KVM/VMware	Hyper-V	Xen	VirtualBox	Others ²
•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•
•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•

Soussi et al. [28]

Our work

¹Hyper-V/C/R, Wharf, Red Hat OpenShift, Fedora C/R, H-Container, Heterogeneous Container Migration (HETCOM), Fault Tolerance Interface (FTI), Process Hauler (P.Haul), Berkeley Lab Checkpoint/Restart (BLCR)²OpenStack, Red Hat OpenShift, Harvester

integrity over millisecond latency, utilizing migration to checkpoint training progress and resume on healthy nodes without restarting the entire epoch Imran et al. [3]. Similarly, in IoT and Edge computing, the objective often shifts to mobility support, where services migrate to follow physical users to minimize network latency Imran et al. [3]. Finally, an interactive application like a Jupyter Notebook might tolerate a suspension of several seconds, or even minutes, as the human user's reaction time buffers the delay Imran et al. [3].

This variance highlights that the classification of migration as “live” or “cold” is dictated by the acceptable downtime for a given use case rather than a rigid technical threshold. Classifying service interruption is highly context-dependent, varying according to individual use cases, technical intricacies, and performance metrics unique to each scenario. As a result, creating a universal definition of live migration with fixed, standardized metrics that applies to all situations is impractical. Ultimately, if the downtime falls within the acceptable limits defined by the user's SLA, the migration is functionally considered “live” Kaur and Chana [2].

Scope of the work and relevant studies

The ISO 13008 standard defines the term migration as the transfer of records from one hardware or software configuration to another, ensuring the preservation of their original format through a well-defined and structured set of principles ISO 13008 [8]. After migration, the service or software component is required to comply with precise specifications and rigorous quality evaluation criteria to ensure optimal performance and reliability¹. These criteria play a vital role in guaranteeing that the migrated entity maintains its functionality, performance, and reliability within the new environment, ensuring the integrity and usability of the transferred records or applications remain intact. In the context of process migration, three primary categories are typically identified: no migration, cold migration, and live migration. In the first category, the process remains on its original host, while in the subsequent two categories the process will be moved to another host. Live migration expands the concept of traditional migration by enabling the seamless transition of a running computational entity, such as a VM or container, from one host to another with little to no service disruption. This stands in contrast to cold migration, which allows service disruption during the migration from the source to the destination.

Live migration performance is measured by (1) migration time – the total duration from the start to the end of the migration process – and (2) migration downtime

¹For example, the ISO 25010 and ISO 25011 standards ISO 25011 [143], ISO 25010 [144].

– the period of service unavailability Sarker and Tang [9]. The length of downtime determines whether migration is classified as live or cold, depending on its acceptability for a given use case. Classifying service interruption is also highly context-dependent as it varies according to individual use cases, technical intricacies, and performance metrics unique to each scenario. As a result, creating a universal definition of live migration with fixed, standardized metrics that applies to all situations is impractical Liu and Fan [10], Rovnyagin et al. [11].

An overview of the existing literature on live migration is presented in the rest of this section and summarized in Table 1. Emphasis is placed on recent studies that examine live migration from technical standpoints. Kaur et al. [25] proposes a taxonomy of techniques for live container migration, focusing on various methods such as pre-copy and post-copy migration, prediction models, and bandwidth optimization. It explores container migration schemes across three computing layers: cloud, Fog, and Edge computing. A similar study He and Buyya [27] offers a comprehensive overview with multiple management-related categories and subcategories concerning live migration in Edge and cloud computing environments.

From an energy optimization perspective, particularly for data centers in cloud environments, Soma and Rukmini [5] provides an overview of research conducted on both containers and virtual machines. The authors summarize key studies on live migration in these environments, outlining individual methods and results.

On the other hand, Imran et al. [3] classifies live virtual machine migration techniques from multiple perspectives: load balancing, energy efficiency, SLA awareness, and network-awareness. They provide a thorough discussion of various approaches within these categories, including push-pull strategies, hybrid bio-inspired algorithms, AI-based methods like Backpropagation Artificial Neural Network for load balancing, energy-aware schemes such as Efficient Adaptive Migration Algorithm and Discrete-Time Markov Chain (DTMC), and SLA-aware techniques that utilize reinforcement learning and neural networks. The study effectively surveys various live VM migration methods and compares their performance across multiple metrics. A growing number of review papers, such as Ramanathan et al. [24] conduct comparative analysis of different migration methods in Fog computing, focusing on key metrics such as total migration time, downtime, and the overall volume of transferred data.

In summary, while the existing literature on live migration covers generic aspects mainly within the scope of cloud computing environments and management-centric perspectives, this work extends the scope beyond these and provides a comprehensive technical evaluation of the

comparative advantages, disadvantages, and trade-offs among different technologies, offering detailed comparisons and in-depth analyses of live migration solutions. Furthermore, given the rapid evolution of the live migration field, regular updates are necessary to incorporate the latest advancements in live migration tools and technologies.

Migration techniques

Live migration primarily relies on shared storage due to its efficiency in transferring only process memory and state between hosts, reducing migration time and network load, while enabling simplified management. However, alternative methods like “shared nothing” live migration are also available to increase flexibility Soussi et al. [28]. In shared storage live migration, only VM memory, CPU, device, and network states are transferred, while shared nothing migration requires transferring the entire VM storage in addition to the memory and states. Data migration across hosts can be achieved via two primary approaches, distinguished by the order in which key components are transferred. The first approach prioritizes the transfer of the process’s memory content before moving other elements, while the second method initially transfers the process state, followed by other components. Each of these approaches offers distinct advantages and trade-offs depending on the specific use case and system requirements.

Pre-copy

The pre-copy policy begins by transferring all memory pages and required files to the destination host while the process remains operational. Subsequently, it iteratively transfers the modified data (dirty pages) during each iteration until the amount of modified data becomes minimal or satisfies predefined termination conditions. At this point, the process is paused, the remaining dirty pages and state information are migrated, and finally, the process is restored to continue operation on the destination host He and Buyya Rovnyagin et al. [11], [27]. Live migration via pre-copy introduces challenges, such as limited parallelism during data dumping, non-convergence of the pre-copy strategy, and insufficient parallelism in the recovery process. Recent studies aim to address these challenges by employing enhanced page servers, memory activity detection, Auto-converge, and machine learning-based behavior predictions Zhang et al. [30], Haris et al. [29].

Post-copy

This approach transfers the process’s state to the target host and resumes it there before transferring the memory. Memory pages are then actively pushed from the source to the target, and any missing pages are fetched

on demand. Strategies such as ensuring that each page is transferred only once, significantly reduce the copy redundancy compared to pre-copy method He and Buyya [27], Hines and Gopalan [31]. The post-copy method however, imposes a high risk of data loss if a failure occurs before all memory pages are transferred. Therefore, although post-copy introduces less overhead comparing to pre-copy, and different variations of post-copy such as demand paging (lazy copy), active pushing, pre-paging, and dynamic self-ballooning Hines and Gopalan [31] have been proposed, almost no real-world computing system adopts post-copy migration Puliafito et al. [32] due to its lower reliability.

Considering the advantages and disadvantages of both pre-copy and post-copy methods, various hybrid methods were proposed to combine the strength of both methods. Hybrid methods aim to transfer memory via pre-copy, and process's state via post-copy, thus enhancing both reliability and efficiency He and Buyya [27], Soussi et al. [28], Nayak et al. [33]. When the characteristics of a workload are predictable, more advanced variations such as speculative method Lu et al. [34] can be employed to further improve the overall performance.

It is worth emphasizing that the boundaries of live migration can only be defined in combination with a concrete use case. This context-dependent nature allows even simple methods like stop-and-copy to be considered viable for live migration in specific scenarios; for instance, in interactive applications where migration downtime falls below the required interaction timeout.

Migration can be performed at various levels of granularity, ranging from page-level (smaller granularity) to the migration of an entire node (larger granularity).

The selected level of granularity plays a critical role in determining the overall performance and efficiency of the migration process. Larger granularity can distribute migration overheads more effectively, but it risks wasting fast memory capacity if the application does not fully utilize the migrated data. On the other hand, smaller granularity provides finer control over the process, though it may lead to increased overhead due to the need for more frequent transfer. Existing studies and empirical observations suggest that while finer granularity may provide more precise control, the current state of technology, tooling, and industry practices favors VM and container-level migration as the most practical and beneficial approach. This is due to several key factors, including portability and flexibility, resource efficiency, scalability and management, development and deployment speed, application isolation, ecosystem support, and modernization pathways. Therefore, this work focuses on live migration strategies at the VM and container level.

Checkpoint/restore tools

To implement these migration techniques, various C/R tools have been developed to capture the state of a running process (checkpointing) and later recreate it as a new running process (restoring), effectively duplicating the original process context. C/R tools can operate at different levels, namely: system-level (e.g., BLCR) executed in kernel space, user-level (e.g., Distributed Multi-Threaded CheckPointing (DMTCP), Checkpoint/Restore In Userspace (CRIU)) executed in user space, and application-level (e.g., FTI) requiring modifications to the application code Jia et al. [36], Benjaponpitak et al. [35]. Each C/R tool pursues distinct goals and techniques, and the execution environment imposes different constraints. The rest of this section explores the technical characteristics of some the most prominent C/R tools, listed in Table 2.

Berkeley lab checkpoint/restart

BLCR was a system-level checkpoint/restart implementation for Linux clusters targeting HPC applications, though it is no longer in active maintenance. It had a hybrid kernel/user implementation that provides checkpoint/restore functionality without requiring changes to application code. BLCR integrated with various MPI implementations like MVAPICH and OpenMPI, and based on its extensions, various supporting tools such as a middle-ware for load balancing between the nodes had been proposed Gerofi et al. [37]. However, due to BLCR's lack of support for SysV shared memory, these implementations were unable to continue supporting BLCR Xu et al. [38].

Table 2 Overview of Checkpoint/Restore tools

C/R Tools	Migration Tech.	Execution Env.	Limitations	Use cases
BLCR	Pre-copy	Kernel space	Linux specific	HPC
DMTCP	Pre-copy	User space	Partial state	HPC
CRIU	Pre-copy/Post.	User space	Linux specific	Container
P.Haul	Pre-copy/Post.	User space	Overhead	Container
FTI	Pre-copy	App. code	Code modification	HPC
OpenShift	Pre-copy/Post.	OpenShift env.	High Granularity	Container
Fedora C/R	Pre-copy	User space	Platform specific	Process
Hyper-V C/R	Pre-copy	Hypervisor	Windows specific	VM
CAST AI	Pre-copy/Post.	Clouds	Platform specific	Cloud
H-Container	Pre-copy/Post.	User space	Linux specific	Container
HETCOM	Pre-copy	User space	Linux specific	Security
Wharf	Pre-copy	User space	Adoption	Application

Distributed multithreaded checkpointing

To use DMTCP, a process or container must be initiated with a predefined library linked from the start, which could be regarded as a limitation Poggiani et al. [39]. Additionally, it intercepts the process's glibc/kernel calls to create a checkpoint, however it is sometimes unable to capture certain calls, such as `inotify()` which might lead to inconsistencies or data loss Poggiani et al. [39]. Comparing the performance of BLCR and DMTCP Azeem and Helal [40] concludes that BLCR has a weaker performance in terms of time needed for checkpointing and restarting. DMTCP requires a separate plugin for each network interconnect, making it costly to maintain and not fully network-agnostic. To address this, the open-source tool MPI-Agnostic Network-Agnostic Transparent Checkpointing (MANA) was proposed, using a split-process mechanism that allows to checkpoint/restart only the part of the process not belonging to the MPI library while tearing down and reinitializing the MPI part, relieving this limitation Xu et al. [38].

DMTCP is a user-space tool for checkpoint/restart of multi-threaded, distributed applications without kernel modifications that also integrates with HPC environments Garg et al. [41], Timalisina et al. [42], Xu and Cooperman [43]. To use DMTCP, a process or container must be initiated with a predefined library linked from the start; it intercepts the process's glibc/kernel calls to create a checkpoint and handles a wide range of system artifacts. Comparing the performance of BLCR and DMTCP, Azeem and Helal [40] concludes that BLCR has a weaker performance in terms of time needed for checkpointing and restarting. However, the requirement for a predefined library could be regarded as a limitation, and it is sometimes unable to capture certain calls, such as `inotify()`, which might lead to inconsistencies or data loss Poggiani et al. [39]. Additionally, DMTCP requires a separate plugin for each network interconnect, making it costly to maintain and not fully network-agnostic. To address this, the open-source tool MANA was proposed, using a split-process mechanism that allows to checkpoint/restart only the part of the process not belonging to the MPI library while tearing down and reinitializing the MPI part, relieving this limitation Xu et al. [38].

Checkpoint/restore in userspace

Originally released in 2012, CRIU relies on Linux kernel features available in kernels since version 3.11. CRIU pauses the migrating process using `ptrace()`, which freezes it in place. Then it gathers all necessary process data from existing kernel interfaces in userspace and writes them as a condensed binary image to disk. Next, parasite code is injected via a Remote Procedure Call (RPC), allowing CRIU to swiftly write all process memory pages to disk. The parasite code is then removed, leaving

the process unaware it was ever injected. The Page-Server mechanism in CRIU reduces filesystem overhead during migration by directly transferring a process's memory content from the source node to the destination node. Instead of storing dump files locally and then transferring them, the Page-Server, running on the destination node, can receive the memory data directly, minimizing storage needs on the source node Pickartz et al. [44]. CRIU supports incremental memory checkpointing capturing and saving only state changes across multiple intervals to minimize checkpoint size and reduce downtime Singh et al. [45]. It is mainly known for its effective container migration capabilities He and Buyya [27]. It has been integrated into different container engines such as OpenVZ, Podman, Borg (Google Container Engine), LXC/Linux Container Daemon (LXD), Docker and others. Practical application of CRIU including successful live migration, rapid initialization, and its integration into Kubernetes (an open-source platform for automating container orchestration and management) has been confirmed in practice Eskandani et al. [46]. Furthermore, empirical studies demonstrated the potential benefits of CRIU in Edge computing, particularly in conjunction with Docker's experimental features, as well as its impact on container performance Tošić [47]. It can be concluded that CRIU in its current state is viable for individual containers when time constraints are not a primary concern, allowing applications with flexible time requirements to benefit from its functionality without significant impediments.

Process hauler

Built on top of CRIU, P.Haul emerged to address a specific limitation of CRIU. While CRIU provides the basic functionality needed for live migration, such as pre-dumping the runtime state of a process, it lacks a cohesive mechanism to manage and execute all the steps involved in live migration as a unified process. This lack of orchestration makes it challenging to use CRIU directly for live migration, especially for complex systems requiring multiple stages of state transfer. P.Haul addresses this gap by acting as a management layer over CRIU. It coordinates the various phases of live migration, particularly for scenarios where pre-copy migration techniques are required. To the best of our knowledge, no significant research work has been published solely on P.Haul. Nonetheless, OpenVZ and LXC attempted to make use of P.Haul Ma et al. [48], Qiu et al. [49]. However, since P.Haul performs image migration by synchronizing the (rootfs), when it comes to OCI² compatible images, it is unable to take

²Standardized container images that adhere to the Open Container Initiative Image Specification. These images contain a set of read-only filesystem layers that form the basis for the container's runtime environment.

advantage of the layered structure of images to prevent the transmission of redundant layers Xu et al. [50].

Fault tolerance interface

Designed for HPC environments, FTI is a user-level checkpoint/restore library Bautista-Gomez et al. [51]. It allocates host buffers for checkpoint data and thus can synergize with live migration techniques. Coupled with resource managers, it offers transparent operation which forms a comprehensive solution for resilient and flexible HPC environments. Both FTI and DMTCP are designed for HPC environment. However, due to seamless integration with resource managers like Slurm, and Remote Direct Memory Access (RDMA) network compatibility, DMTCP outperforms FTI as a versatile solution in live migration, in particular for HPC environments.

Platform specific tools

In addition to the aforementioned tools, several platform specific tools are available. *Red Hat OpenShift* supports live migration for both containerized and VM-based workloads. In terms of containers, OpenShift supports stateful application workload migration within or between OpenShift clusters via Migration Toolkit for Containers (MTC) at the granularity of a namespace, migrating all namespace-level objects rather than just one container Migration-Toolkit-for-Containers [52]. *Fedora Checkpoint Restore* enables transparent OS-level migration of processes, process trees, or containers between Fedora systems without service interruption or application modifications. It utilizes CRIU and is implemented through the crtools package and specific kernel configurations. *Hyper-V Checkpoint Restore* is a feature in Microsoft's virtualization platform that enables saving and restoring virtual machine states, supporting both memory state and data-consistent captures. Combined with live migration capabilities, Hyper-V C/R allows for seamless transfer of running VMs between hosts.

Migration unit

The migration unit is the computational entity transferred between hosts during a migration. As discussed in Sect. 3, in most scenarios, this unit is optimally either a container, a group of containers (pod), or a virtual machine, each requiring distinct approaches and tools for effective migration. The selection of the migration unit affects the complexity, resource requirements, and performance characteristics of the migration process, thereby shaping the overall strategy and implementation of live migration in diverse computing environments. The decision made here influences and is shaped by several key elements, including the volume of data to be transferred, the time required for the process, and system compatibility.

Container

In the modern computing landscape, containerization has revolutionized software application development, distribution, and deployment. This approach involves packaging an application with its essential components, including binary code, libraries, dependencies, and configuration files into lightweight, portable, and consistent environments known as containers Gamess and Parajuli [53] which generally incur less virtualization overhead compared to virtual machines Govindaraj and Artemenko [54].

The container landscape primarily consists of application containers and system containers. Application containers are lightweight, standalone packages that encapsulate a single application and its dependencies, while system containers are more comprehensive units and include a full operating system environment, allowing them to run multiple applications or services within a shared kernel space. Both types of containers share the host operating system's kernel. System containers, in addition, provide an isolated user space, offering a complete environment.

In the context of containerization, a container engine typically provides higher-level functionality, including image management, build processes, and user interfaces. The container runtime, on the other hand, focuses on the core tasks of creating and running containers. Given the rapid development and adoption of containerization concept in research and industry, the terminology isn't always consistent, and terms are often used interchangeably or with slightly different meanings depending on the context. Therefore, this work uses "container technology" to refer to both container engine and runtime, unless a specific distinction is needed. Table 3 provides a list of container technologies categorized based on their main functionality.

Among the largely adopted container technologies, Docker and Podman primarily focus on application containers, while LXC is designed for system containers Silva et al. [55], and Apptainer targets HPC environments. A growing standardization effort aims to improve interoperability among container technologies, enabling interchangeable components across frameworks and better compatibility across computing systems. Consequently, the boundaries between existing technologies are often blurred, making it difficult to draw clear distinctions in many cases.

In the following section, the most relevant container engines with respect to their support for live migration are discussed.

Linux container

A system environment close to a full Linux system is provided by LXC by launching an OS in every container.

Table 3 Containerization and orchestration technologies

Category	Technology	Description
Engine	Docker	Daemon-based
	Podman	Daemonless
	LXC	OS-level
Runtime	Apptainer (aka Singularity)	HPC-focused
	Containerd	Low-level
	CRI-O	for Kubernetes
Self-hosted Orchestration	runc	Low-level
	Kubernetes (K8s)	Large-scale
	Docker Swarm	Small-scale
	Hashicorp Nomad	Large-scale / Mixed workloads
	Apache Mesos	Large-scale resource scheduler
Managed Orchestration Service	Google Kubernetes Engine (GKE)	Managed K8s by Google Cloud
	Amazon Kubernetes Service (EKS)	Managed K8s by AWS
	Azure Kubernetes Service (AKS)	Managed K8s by MS Azure
	Oracle Kubernetes Engine (OKE)	Managed K8s by Oracle Cloud
Specialized	Red Hat OpenShift	Enterprise Kubernetes
	MicroK8s	Kubernetes for IoT
	K3s	Kubernetes for IoT

Soussi et al. [28], Silva et al. [55], LXC-Introduction [56]. LXC leverages the namespace mechanism to isolate containers and utilizes native Linux Control Groups (CGroups) to share the kernel with containers while managing CPU, memory, disk and network resources. The concept of LXC makes it in many directions comparable to VMs Kaur et al. [25]. LXC integrates CRIU, however, it does not yet support checkpointing of network namespaces Das and Sidhanta [57]. An enhanced and user-friendlier version of LXC is LXD that is built around REST APIs to communicate with LXC software library (liblxc), enhancing the LXC features such as improved security and isolation, live migration and limiting resource of the containers Qiu et al. [49], Silva et al. [55], Vazquez [58].

Docker

Initially built on LXC, Docker later transitioned to its own container runtime library, libcontainer, to achieve greater control and flexibility. Silva et al. [55]. Studies show that on average less than 7% of the content inside a Docker image is used during the runtime, thus, the layered property of Docker images has been leveraged to further reduce the migration overhead Harter et al. [59]. A similar study found that each Docker image layer has a unique SHA256 local ID and proposed a method to prevent redownloading layers with different SHA256 IDs. Other works such as Sledge Xu et al. [50]— a live

migration system, are focused on fine-tuning the framework by introducing Lightweight Container Registry (LCR) for container images, utilizing CRIU's incremental memory checkpointing ability for an improved runtime, and by designing Dynamic Context Loading (DCL) scheme for effective management. Voyager Nadgowda et al. [60], a just-in-time migration tool built on CRIU, that minimizes downtime by employing data federation via union mounts. It utilizes tmpfs for dump files to avoid storage latencies Cao et al. [61] and adopts a multi-pronged migration approach: CRIU handles in-memory state transfer, a dual-band system manages local filesystem migration between source and target, and network filesystems are migrated by swapping mount points. This tool is highly dependent on network-attached storage and upon any degradation of network quality, the migrated service may experience frequent errors in the background-replicated files, leading to a general performance degradation Nath et al. [62]. Other research efforts focus on minimizing data transmission by optimizing Docker's storage mechanisms and employing algorithms such as Joint Network and Local Prediction (JNLP) to reduce dirty memory pages Wang and Xie [63]. Experimental results demonstrate decreased migration times for both stateless and stateful containers.

Podman

As a daemonless, Linux-native, open-source tool developed by Red Hat, Podman is built on runc and allows users to create OCI-compliant containers. It is based on libpod, a library for container lifecycle management and provides APIs for managing containers, pods, container images, and volumes [64]. Podman's daemonless design allows containers to run rootless by default, eliminating the need for background services such as Docker's dockerd. Podman also integrates CRIU with pre-copy optimization, enabling checkpointing and restoring containers across hosts. Although based on our practical experience, stateful checkpointing and restoring of Podman containers works smoothly on Red Hat-based distributions, containers relying on a systemd-based entry point may encounter issues during checkpointing since CRIU cannot fully capture systemd's state, and active TCP connections require the *-tcp-established* flag for the connections to be properly handled. Several studies also note that performing reliable live migration via Podman requires root permission Podman-GitRepo [65], Podman-Checkpoint [64]. Except for Podman, which requires CRIU to skip network namespace recreation while restoring a checkpointed container, other container technologies let CRIU handle namespaces Reber [66], Podman-Checkpoint [65].

Apptainer

Previously known as Singularity, it was originally developed for HPC ecosystems. Apptainer container engine offers features such as native support for GPUs, MPI, InfiniBand, and resource managers like Slurm Apptainer [67]. Unlike Docker and LXC, it does not rely on CGroups which means less strict resource constraints, and operates without root daemon. In contrast to Docker and Podman, Apptainer does not provide built-in support for CRIU Andrijauskas et al. [68]. One reason is the single-file image format of Apptainer in contrast to the layered file systems, which makes it an undesirable candidate for the incremental checkpointing feature of CRIU. Besides that, Apptainer bypasses the host library stack, preventing the preloading of libraries required to enable checkpointing. There has been various efforts to enable checkpointing for Apptainer containers using DMTCP DMTCP-Singularity-Plugin [69], MWE-Singularity-checkpoint, [70]. The required checkpointing libraries could also be manually injected via an specfile to realize a checkpointable Apptainer container. Nevertheless, Apptainer has recently integrated support for DMTCP, enabling experimental checkpoint and restart functionality for Apptainer containers. However, according to Apptainer's documentation, this capability remains in an experimental stage and is not yet considered production-ready [71].

Virtual machine

As a well-established technology, VM live migration enables the seamless transfer of running VMs between

physical hosts without service interruption. Despite challenges like managing large memory states and network connections, the technology has evolved to enhance resource utilization and system reliability. As virtualization continues to advance, VM live migration capabilities are becoming increasingly efficient, addressing more complex scenarios and larger VM instances. Table 4 provides a list of VM technologies categorized based on their main functionality.

Majority of production VM live migration implementations rely on hypervisors, which utilize pre-copy and post-copy techniques. Hypervisor provides hardware virtualization capabilities and is categorized into two types. Type 1 hypervisors, also known as bare-metal hypervisors, interact directly with the host machine's physical hardware, bypassing the need for an underlying operating system. This direct access allows for more efficient resource allocation and management, resulting in optimal performance and reduced overhead. In contrast, Type 2 hypervisors, also called hosted hypervisors, run as applications within an existing operating system. They access hardware resources through the host OS, which can lead to lower performance compared to Type 1 hypervisors due to the additional software layer.

VMware vSphere

The vSphere platform, with its Type 1 ESXi hypervisor and vCenter Server for management, supports application virtualization, high availability, and disaster recovery, while optimizing performance, security, and deployment Patrão [72], VMware vSphere vMotion enables live migration of workloads between hosts. Unlike vSphere High Availability (HA), which automatically restarts VMs on other servers after a failure, vMotion focuses on workload balancing and maintenance without requiring VM restarts. Recent studies show that machine learning approaches can effectively improve the overall performance of vMotion in live migration scenarios Elsaid [73].

Microsoft hyper-V

As a Type 1 hypervisor, Hyper-V transfers running virtual machines between physical hosts. Hyper-V works similarly to VMware vMotion, with best practice recommending a dedicated Virtual Local Area Network (VLAN) to isolate migration traffic from application traffic Elsaid et al. [74], VM-Live-Migration-Overview [75].

Xen

As an open-source Type 1 hypervisor, Xen supports VM migration between different Xen-powered servers and enables the execution of multiple operating systems on the same machine. Xen uses the pre-copy mechanism by default, to provide live migration on all versions of Xen-Server Kapil et al. [76]. Migration of Xen VMs to KVM

Table 4 Virtual machine technologies

Category	Technology	Description
Type 1 Hypervisor	VMware vSphere	Enterprise-grade
	Microsoft Hyper-V	Windows Server
	Xen	Open-source
	Kernel-based Virtual Machine (KVM)	Kernel-based
Type 2 Hypervisor	Oracle VirtualBox	Free, cross-platform
	VMware Workstation	Desktop virtualization
	Parallels Desktop	Mac-focused
Cloud based	Amazon EC2	AWS VMs
	Google Compute Engine	GCP VMs
	Microsoft Azure VMs	Azure VMs
	OpenStack	Open-source
Management	VMware vCenter	Centralized
	Microsoft VMMManager	For Hyper-V
	Citrix XenCenter	For XenServer
Emulators	Quick Emulator (QEMU)	Open-source
Specialized	Wind River Hypervisor	Embedded systems
	User Mode Linux	Linux kernel-based
	Red Hat OpenShift	Container and VM
	Harvester	Open-source, HCI

is also possible for various Linux distributions. Since XenServer 6.1, storage and block migration techniques are also supported which enables live migration of VMs without shared storage. Xen supports two virtualization types: paravirtualization (PV), where guest OSes interact directly with the hypervisor for resources, and Hardware Virtualized Machines (HVM), where unmodified OSes run with virtual hardware emulation, resulting in higher overhead. Vazquez [58], Xen-Project-Beginners-Guide [77].

Kernel-based virtual machine

KVM, a Linux kernel module enabling hypervisor functionality, has been part of the official kernel since version 2.6.20. KVM supports live migration, while the VM stays powered on, maintains network connectivity, and keeps applications running during relocation. It also enables checkpointing a VM's state for later resumption.

Quick emulator

QEMU can emulate an entire system, including the processor and various peripherals. It supports live migration using the pre-copy approach, where the VM's memory is transferred in multiple iterations while it continues running, followed by a brief pause to send the final dirty pages and CPU/device state, and post-copy approach, where the memory migrates upon the destination VM's Nested Page Table (NPT) faults Storniolo et al. [78], Li and Li [79]. To reduce migration time and downtime in pre-copy, multiple file descriptors (multifd) enables parallel transfer of memory pages using multiple threads Yang et al. [80], Li and Li [79]. The new mapped-ram stream format complements this by assigning fixed offsets to RAM pages in the migration file, allowing concurrent, conflict-free writes and enabling efficient I/O with O_DIRECT (write directly to disk and skip caching) QEMU-Migration [81].

Openstack

As an open-source cloud platform, OpenStack manages compute, storage, and networking resources within a data center. It also allows users to perform live migration of VMs on KVM-libvirt and Xen hypervisors, provided that certain conditions are met across all nodes, such as having a shared instance path across compute hosts, enabling passwordless SSH for root access, and configuring firewalls to allow libvirt communication. If the rate of dirty memory pages is high during live migration, it may fail, but workarounds such as post-copy migration or auto-converge mechanism, which throttles the source worker's CPU to slow it down and keep up with memory changes, are proposed Mirkhanzadeh et al. [82], Configure-Live-Migrations [83]. When using KVM-libvirt, these limitations must be considered, whereas for Xen,

different requirements apply, including the use of compatible XenServer hypervisors and an NFS export visible to all XenServer hosts Configure-Live-Migrations [83].

Hybrid and emerging technologies

As listed in Tables 3 and 4, the containerization and virtualization landscape extends beyond these well-established open-source technologies and includes a diverse range of solutions.

OpenVZ, primarily a container technology, offers features that share similarities with both containers and VMs Walters et al. [84]. OpenVZ has its own set of utilities for managing containers, such as vzctl and prctl Di et al. [85], Vazquez [58]. In 2006, OpenVZ introduced its own live migration approach for containers that later on was replaced by CRIU Checkpointing-and-live-migration [86]. By treating containers as regular processes, OpenVZ reduces management overhead. Comparative studies of container live migration capabilities between OpenVZ and LXC have shown mixed results. While LXC demonstrated shorter total migration times and downtimes, OpenVZ exhibited a less intrusive approach with lower resource utilization during the live migration process. This highlights OpenVZ's efficiency in resource management, despite slightly longer migration durations Indukuri [87].

Oracle VirtualBox is another open-source, cross-platform virtualization software that supports live migration, called "Teleporting". While VirtualBox allows for cross-platform live migrations, its live migration is generally considered less advanced than other virtualization platforms like Hyper-V.

Built based on KVM Habib [88] and KubeVirt Karlsson [89], Red Hat OpenShift also supports live migration for both containers and VMs using shared storage with ReadWriteMany (RWX) access mode, blurring the lines between these technologies and demonstrating overlap in management platforms. Leveraging the Migration Toolkit for Virtualization (MTV), it enables the migration of VMs from other hypervisors (VMware vSphere, RedHat Virtualization (RHV), and OpenStack) into the OpenShift environment Migration-Toolkit-for-Containers [52].

Similarly, evolving Hyper-Converged Infrastructure (HCI) platforms like the open-source Harvester, built on Kubernetes, now run both VMs and containers on bare metal, integrating KubeVirt for virtualization and Longhorn Kampadai et al. [90] for distributed storage. Nevertheless, live migration in these technologies still faces challenges. For instance, migrating virtual machines between nodes with different CPU models remains problematic in certain scenarios Harvester [91].

This convergence within a single platform highlights the evolving cloud-native landscape, where the

distinction between virtualization methods is becoming less defined. Major technology companies including Google, Amazon, Oracle, and Microsoft also offer proprietary solutions in the form of Infrastructure-as-a-Service (IaaS) that address specific enterprise needs. These technologies span cloud computing, containerization, and hybrid environments. Strictly speaking, Amazon Elastic Compute Cloud (EC2), Google Compute Engine (GCE), and Azure are cloud services rather than fundamental virtualization technologies (like KVM, Xen, or QEMU). However, our rationale for including them in Table 4 is that major cloud providers rarely use stock open-source hypervisors “off the shelf.” Instead, they develop highly customized, proprietary implementations, such as Amazon Web Services (AWS) Nitro (a custom KVM-based hypervisor) for EC2 instances or Azure’s specialized version of Hyper-V which include unique optimizations for security Hypervisor-security [92]. A similar distinction applies to managed Kubernetes offerings such as GKE, Amazon K8s Service EKS, AKS, and OKE mentioned in Table 3.

Beyond established platforms, several emerging tools offer promising alternatives, in particular enabling effective migration across heterogeneous platforms. *H-Container* enables the migration of natively compiled containerized applications across compute nodes with CPUs of different Instruction Set Architectures (ISA) Barbalace et al. [93]. It is built upon CRIU and extends Popcorn Linux to a container environment Bapat et al. [94]; it uses Intermediate Representation (IR) lifting software to instrument applications for cross-ISA migration without requiring the source code Barbalace et al. [93]. *HETCOM* focuses on secure container migration in heterogeneous computing environments; it is not bound to any container engine and does not rely on any existing C/R tool. It ensures security by using Trusted Execution Environment (TEE)s on edge nodes and using Trusted Platform Module (TPM) 2.0 on cloud nodes, aiming for seamless integration with emerging technologies with low overhead Wruck et al. [95]. *Wharf* introduces a new paradigm for transparent and efficient live migration across heterogeneous hosts. It uses an abstraction called “vessel” to provide a machine-independent representation of an application’s state, and “dock” as a runtime layer for executing and migrating vessels, allowing transparent live migration between different ISAs and operating systems. The authors assert that Wharf’s checkpoint and restore time is, on average, 14× faster than CRIU Yang et al. [96].

In parallel with these developments, specialized paradigms are emerging to address specific efficiency and security needs. Unikernels represent a specialized virtualization paradigm where the application is compiled with a minimal OS kernel into a single-purpose

appliance. Unlike full-fledged VMs, Unikernels eliminate general-purpose OS overhead, resulting in a significantly reduced attack surface, faster boot times, and lower memory footprints ideal for resource-constrained Edge and IoT devices Chen and Zhou [97]. For live migration, this lightweight nature is a major advantage. Studies demonstrate that Unikernels can be migrated with shorter total migration time compared to Docker containers in highly mobile environments, such as vehicular networks Chen and Zhou [97]. Furthermore, while standard Unikernels are stateless, novel frameworks like MirageManager introduce checkpoint-based live migration by preserving state at the function level Cozzolino et al. [98]. When integrated with serverless architectures (e.g., UKFaaS in Chen et al. [99]), Unikernels offer secure, high-performance isolation, making them a promising candidate for secure, low-latency migration.

On the other hand, the shared host OS architecture exposes containers to significant security challenges. Specifically, they are vulnerable to Denial of Service (DoS) attacks if a single application monopolizes resources, as well as critical kernel exploits where compromising the host OS grants an attacker access to all running applications Mansouri and Babar [100]. In contrast, VMs benefit from stronger isolation mechanisms that mitigate these shared-kernel risks. This protection is further enhanced by advanced security measures, such as blockchain-based integrity verification, which effectively shields VMs against external attackers Imran et al. [3]. Recent advancements in VM-based confidential computing technologies, such as Intel Trust Domain Extensions (TDX), AMD Secure Encrypted Virtualization with Secure Nested Paging (SEV-SNP), and IBM Secure Execution, protect entire virtual machines with hardware-enforced memory encryption and integrity, in contrast to embedded solutions like ARM TrustZone Confidential-Kubernetes [101]. To enable secure live migration while limiting trust in the hypervisor in these environments, additional architectural components are required to handle attestation and key movement without exposing guest data. Intel TDX supports migration of isolated “Trust Domains” (TD) and can employ a dedicated trusted migration domain (often referred to as Migration Trust Domain (MigTD)) to enforce migration policies and mediate the transfer of encryption keys for these TDs Kollenda [102], Intel-TDMig [103]. Similarly, AMD SEV-SNP uses a centralized Migration Agent (MA) that coordinates migration policies and re-encryption of guest state during transfer Soussi et al. [28] Kollenda [102]. While these VM-level TEE technologies simplify secure execution compared to legacy enclave-based APIs like Software Guard Extensions (SGX), migration-oriented components such as MigTD and standardized MA-driven workflows are still maturing and are

Table 5 Comparative overview of live migration support by prominent technologies

Technologies	CRIU-based ¹	HPC/DMTCP-based ²	Open-VZ	VMware vMotion	KVM/QEMU
Migration unit	Process or Container	Process (across machines)	Full VState	VM	VM
Implementation	Kernel level(ptrace API)	Library level(intercepts calls)	Kernel extension	Hypervisor-based	Hypervisor-based
Prerequisite	Recent kernels	Special launch script	Open-VZ kernel	VMware	KVM support
Userspace vs. Kernel	Requires kernel modules	Runs in userspace	Kernel-based	Hypervisor-based	Hypervisor-based
Platform integration	Docker and Kubernetes	No	No	No	Yes
App compatibility	Wide range	Limited	Wide range	Wide range	Wide range
App transparency	Yes	Yes	Yes	Yes	Yes
User transparency	No	Partial	Yes	Yes	Yes
Distributed Apps	Limited	Strong support	Across hosts	Long-distance	Across hosts
HPC relevance	Limited	Yes	Yes	Yes	Yes
Overhead	Generally lower	Issues due to proxying	Minimal	Minimal	During migration
PID handling	Maintains original PIDs	May use fake PIDs	Maintains PIDs in VE	N/A (VM-level)	N/A (VM-level)
Net connection	No open connection	Yes	Yes	Yes	Yes

¹The CRIU-based column includes technologies such as P.Haul, which are based on CRIU

²The DMTCP-based column refers to solutions, which are based on DMTCP

not yet uniformly deployed across production environments Soussi et al. [28].

This diversification reflects the ongoing innovation driven by increasing demands for scalability, performance, and security across various computing environments. Table 5 presents a comparative overview of the key features of prominent containerization and virtualization technologies. As shown in this Table, while process-level solutions such as CRIU and DMTCP offer fine-grained migration capabilities, they are limited in user transparency, platform integration, and support for distributed or HPC workloads, with DMTCP providing stronger support for distributed applications at the cost of increased overhead Timalsina et al. [42]. OpenVZ

presents a balanced approach with low overhead and good user transparency but suffers from limited modern platform integration due to its reliance on specialized kernels Checkpointing-and-live-migration [86]. In contrast, hypervisor-based solutions like VMware vMotion and KVM/QEMU stand out for their robust, seamless, and highly transparent live migration at the virtual machine level, offering minimal overhead, broad application compatibility, and strong integration with contemporary cloud and enterprise environments [83]. Ultimately, the choice of technology should be guided by the required migration granularity, application type, and operational environment, with VM-level solutions providing most effective for general-purpose and cloud workloads, and process-level tools remaining relevant for specialized scenarios such as HPC and tightly-coupled distributed systems.

Infrastructure characteristics

Infrastructure characteristics, including network conditions and resource availability, directly affect live migration's performance, efficiency, and reliability. These characteristics, combined with environmental preferences and constraints, influence the overall strategy for live migration.

Network conditions and resource availability

Live migration is strongly influenced by both network conditions and resource availability, as the process involves substantial data transfer and coordination across heterogeneous infrastructures. Network characteristics become particularly critical when transitioning from Local Area Network (LAN) to Wide Area Network (WAN)s, where lower bandwidth, higher latency, congestion, and the need for IP reassignment introduce significant challenges that can extend migration times, increase downtime, and complicate reconfiguration, especially when it requires transferring large volumes of data in shared-nothing setups. In particular, the total migration time and application dwell time (downtime) are tightly coupled to the available bandwidth and the rate at which memory and storage pages are dirtied during execution Choudhary et al. [104], Haris et al. [105]. In LAN environments with shared storage such as Storage Attached Networks (SAN), migration can often be simplified to memory-state transfer, whereas the absence of shared storage in WAN environments necessitates full disk transfers, intensifying bandwidth saturation and increasing the likelihood that pre-copy migration fails to converge when available bandwidth cannot outpace the memory dirtying rate, thereby requiring adaptive termination conditions or hybrid strategies Hosseini Shirvani et al. [106], Haris et al. [105]. High latency and congestion further affect service continuity and application

stability, while simultaneous migration of multiple virtual machines strains network resources and demands efficient scheduling and bandwidth utilization. Additionally, WAN-based migration carries an elevated risk of data loss due to network failures, which can be mitigated through replication and erasure coding, and optimized through techniques such as deduplication, compression, or post-copy variants incorporating pre-paging and active push mechanisms to reduce page faults and initial downtime Choudhary et al. [104], Hosseini Shirvani et al. [106]. Alongside network constraints, the availability and allocation of computing resources, including CPU/GPU, memory, and storage, are equally decisive, as sufficient processing and memory capacity are required to execute migration tasks efficiently while maintaining acceptable performance on both source and target hosts Widjajarto et al. [107], and adequate storage capacity and performance are necessary to accommodate migrated data and enable smooth transfer of large datasets Govindaraj and Artemenko [54]. Moreover, live VM and container migration is tightly coupled with hardware-level compatibilities; configurations such as CPU pinning can bind a VM to a specific host unless the target provides identical hardware and passthrough identifiers, and a lack of hardware homogeneity across clusters obliges infrastructure-level coordinators to perform complex compatibility checks on ISAs to avoid post-migration application crashes. Consequently, migration decisions inherently involve trade-offs between migration speed, network utilization, and service continuity, and must account for both environmental constraints and underlying hardware dependencies, underscoring the complexity of WAN-based live migration and the need for careful planning and optimization strategies Choudhary et al. [104], Haris et al. [105].

Cluster environment

Clusters, in classic terms, are a type of computing where multiple computers are connected to work together as a single system. They are typically designed to provide high computational power and are often used for specific tasks or workloads that require significant processing capabilities.

A prime example of clusters is HPC clusters, which aggregate computing resources to tackle complex, large-scale workloads. In HPC clusters particularly, GPUs play a crucial role. Although C/R for GPU-enabled VMs is possible, since GPUs maintain extensive internal states, it is significantly more challenging compared to CPU-enabled, and highly depends on the specific technology and setup Russel [108]. NVIDIA vGPU supports live migration via mediated passthrough when using NVIDIA GPUs with vGPU technology Xu et al. [109]. Hyper-V in Windows Server 2025 supports live migration with GPU

partitioning (GPU-P), although there are reported difficulties in its implementation. gMig reported successful live migration for full virtualization on Intel GVT-g GPUs with an average downtime of 302 ms on Windows and 119 ms on Linux operating systems Tian et al. [110], Ma et al. [111], Belkhiri and Dagenais [112]. However, it only supports Intel GPUs, and the migration process must wait for GPU tasks to complete before migration, making this approach impractical for long-running GPU workloads. PARALLELGPUOS (POS) is a system designed to improve GPU checkpointing and restoring by enabling concurrent execution of C/R with GPU tasks Huang et al. [113]. However, periodic checkpointing of GPU memory can lead to significant overhead as examined by other studies Han et al. [114]. NVIDIA Checkpoint/Restart (NVCR) is an extension for system-level checkpoint software like BLCR and DMTCP, enabling transparent checkpointing and restarting of CUDA applications without requiring modifications from developers Nukada et al. [115].

As mentioned earlier, CRIU although being the de-facto choice for C/R, originally struggled with tasks involving device states managed by drivers. While CRIU offers a plugin mechanism to address this limitation, GPU support was historically limited. To enable checkpointing for GPU workloads, AMD developed the “criu-amdgpu-plugin” for ROCm applications and NVIDIA released “CRIUgpu” for CUDA applications, both capable of saving and restoring GPU execution states Stoyanov et al. [116], Supporting-ROCm-with-CRIU [117].

Similar to GPUs, RDMA technology introduces complexities for live migration because it utilizes kernel-bypass mechanisms, storing connection states directly on the RDMA Network Interface Controller (RNIC) rather than in the operating system. To overcome this, device-assisted solutions have been developed for specific hardware, such as NVIDIA ConnectX, which utilize a generic device-hypervisor interface to pause and export hardware states, achieving sub-second downtimes with no runtime overhead Polyakov et al. [118]. In contrast, software-based approaches like MigrRDMA implement an indirection layer within the RDMA library to maintain state consistency for containers on commodity hardware, though this incurs a slight overhead Li et al. [119]. Alternatively, OS-level architectures like MigrOS restore migration capabilities by modifying communication protocols, effectively reducing latency compared to pure software virtualization techniques Planeta et al. [120].

Finally, specialized hardware further constrains migration decisions. GPU-enabled workloads require careful consideration of resource locality, performance continuity, and hardware compatibility, imposing a trade-off between the overhead introduced by migration

mechanisms and the importance of the migration objective itself.

Cloud environment

Cloud computing, primarily focuses on providing users with on-demand access to a shared pool of configurable computing resources. Specifically, it emphasizes flexibility, scalability, and the ability to provision resources rapidly Attar-Khorasani et al. [121]. Although traditionally, clusters and cloud computing have had distinct characteristics and purposes, the distinction between clusters and clouds has become less clear-cut in recent years Munhoz and Castro [122], Sherpa et al. [123]. In fact, cloud providers now offer HPC solutions that resemble traditional clusters. In cloud computing environments, specific features such as multi-tenancy and elasticity impact migration. Consequently, shared infrastructure among multiple tenants necessitates careful resource management during migration to maintain isolation and performance Soussi et al. [28]. Furthermore, the ability to scale resources dynamically can facilitate more efficient migrations by allocating additional resources as needed He and Buyya [27]. However, the trade-off between minimizing migration time and ensuring acceptable quality of service is challenging Breitgand et al. [124]. Cloud environments due to their vast amount of resource availability can benefit from certain approaches such as resource reservation for accelerated migration Ye et al. [125], Gupta and Namasudra [126].

Several research works have explored the use of TEEs to enhance the security of container and VM migration in cloud environments. Prior studies show that TEEs provide hardware-enforced isolation that protects sensitive migration operations, such as encryption, integrity verification, and state management, even in the presence of untrusted operating systems or cloud platforms Gao et al. [127], Gao et al. [128]. To address migration consistency and fork/rollback attacks, secure container migration frameworks leverage monotonic counters and ensure existing single-instance at a time during migration Gao et al. [128]. Mutual authentication protocols and encrypted channels along with TEE further ensure the confidentiality and integrity of migrating containers Gao et al. [127]. Beyond containers, TEE-enabled virtualization solutions enable secure live migration of unmodified VMs while maintaining backward compatibility with existing cloud infrastructures Wang et al. [129]. However, recognizing that transferring the entire memory state of VMs or containers can be prohibitively slow for autonomous networks, frameworks like TALOS introduce a finer granularity of migration by targeting only the specific state of Trusted Applications (TA)s-executing inside a TEE-rather than the full execution environment Vasileaidis et al. [130]. These approaches demonstrate that TEE-based

architectures can improve trust, consistency, and isolation in both container and VM migration scenarios.

Although, modern cloud platforms provide support for containers, due to the maturity of VM technology in comparison to containerization, most studies on the topic of live migration consider cloud environments solely consisting of VMs Breitgand et al. [124], Ye et al. [125], Gupta and Namasudra [126].

Nevertheless, infrastructure-level orchestration is helpful in coordinating live migration of both VMs and containers. In cloud-native stacks, Kubernetes with CRI-O as the container runtime (promoted to Beta in v1.30) enables C/R through kubelet APIs, allowing node-level execution without kubectrl and preserving container state during mobility, though it requires persistent storage to avoid data loss in pod recreation. Additionally, KubeVirt extends this unification by enabling seamless VM live migration alongside containers within Kubernetes clusters, supporting hybrid workloads. Despite these advancements, there are only a few recent studies such as a kubernetes-based cloud ecosystem performing migration of Cloud-Native Network Functions (CNF) which focuses on live migration of containerized workloads between remote Kubernetes Clusters Kaur et al. [131]. Though, in this approach, a pod is terminated and then recreated, causing the loss of container state. To address this, preserving container states on a persistent volume, and injecting the state into the new pod upon recreation was proposed, which in turn also reduces the startup time Vasireddy et al. [132]. So despite its progress, the field remains in its early stages and necessitates additional research.

Edge and fog environment

Edge computing and Fog computing are both distributed computing paradigms that aim to bring computation and data processing closer to the source of data generation Govindaraj and Artemenko [54]. While Edge computing is more decentralized and localized, Fog computing creates a hierarchical structure that facilitates data aggregation and preprocessing. Although Fog computing is often used interchangeably with Edge computing, especially in smaller setups; in larger deployments, Fog computing serves as a distinct layer between Edge computing and cloud computing Sahni et al. [133]. The distinguishing characteristics of Edge computing in comparison to cloud and cluster computing are geographical distribution, mobility support, location awareness, proximity, low latency and heterogeneity Khan et al. [134].

Since latency is critical in Edge computing, most live migration research in this field focuses on minimizing downtime, migration time, and overall latency. In addition, Edge computing is closely tied to

resource-constrained servers, which makes live migration primarily container-based.

However, to further overcome the challenges of hardware heterogeneity and limited bandwidth in Edge environments, WebAssembly (Wasm) is emerging as a superior alternative to standard containers. Unlike architecture-dependent VMs or containers, Wasm provides a platform-agnostic execution environment, enabling seamless stateful migration between disparate architectures such as x86-based cloud servers and Arm-based edge devices. Furthermore, Wasm's lightweight memory footprint significantly reduces checkpoint sizes, ensuring faster transfer times and minimal downtime over constrained edge networks compared to traditional virtualization runtimes Fujii et al. [135], Tinto et al. [136].

In contrast, Fog computing, with its resource-rich infrastructure, often involves VM migration Attar-Khorasani and Chalmers [137].

Several recent studies have explored live migration techniques in Fog and Edge computing environments, such as proposing an approach using reinforcement learning for decision-making in live VM migration, considering multiple factors to address early and late handover issues Alqam et al. [138], or introducing *EVM_MIG* system that considers device type, function, and location for VM migration decisions in heterogeneous Fog environments Alqam et al. [139].

Furthermore, Live Migration of Containers in the Edge (LIMOCE), was proposed as a lightweight algorithm for live migration in Edge computing Das and Sidhanta [57]. Other studies proposed smart pre-copy live migration approach for VM migration, aiming to minimize downtime and migration time Osanaiye et al. [140], or enhancement to the post-copy live migration algorithm to make it robust against failures in Fog environments Talebian et al. [141].

Discussion and conclusion

Live migration offers numerous benefits but requires a well-planned strategy. While environment characteristics guide the choice of live migration unit and technique, the ultimate decision depends on migration *objectives* like service availability or energy efficiency, which may sometimes conflict or even discourage live migration altogether.

Across the virtualization spectrum, bare-metal live migration remains impractical, particularly in heterogeneous environments. Conversely, VM live migration technologies have achieved maturity and widespread adoption, offering reliable performance despite higher overheads. Container live migration occupies an intermediate position, providing enhanced speed and reduced overhead compared to VMs while accommodating diverse environments. Although rapidly evolving and

approaching production readiness, container migration continues to face challenges in complex scenarios and is still gaining broader acceptance, especially for latency-sensitive applications. Table 5 offers a general framework for planning live migration strategies. Additionally, several key factors should be taken into consideration. GPU live migration remains a significant challenge for both virtual machines and containers. In scenarios necessitating the preservation of external network bindings, virtual machines generally offer superior capabilities compared to containers. It is strongly advised to utilize recent Linux kernel versions, as leading techniques depend on CGroups functionality and recent system calls. Older kernels lack support for crucial features, such as maintaining PIDs, which are essential for effective implementation of live migration. Most container live migration techniques, relying on CRIU, share common limitations, including inconsistent checkpointing of full process states in complex setups. Using techniques that support OCI-compatible images is recommended, as their layered structure enables efficient incremental migration. In critical environments, live migration may be impractical due to the requirement of root access for most migration techniques. Additionally, on-demand live migration may not be feasible under specific conditions, such as network congestion or when process states are excessively large. It is noteworthy that recent advancements in hardware technologies, particularly RDMA, substantially enhance the broader adoption of live migration techniques by complementing existing software infrastructures, which is particularly beneficial for data-intensive tasks like deep learning, data analytics, and large-scale data transfers.

Container live migration is expected to advance with enhanced hardware support, improved performance for data-intensive tasks, and better integration with cloud-native technologies. Integrating automated DevOps workflows with a data management system and standardized metadata schema enhances data reproducibility, aligns with FAIR³ principles, and adds value by documenting the migration process, data state, dependencies, and Persistent Identifiers Sherpa et al. [142]. Additionally, developments will likely focus on security, Edge-to-cloud continuum support, automated migration decision-making, and improved handling of stateful applications, further expanding its applicability across diverse computing environments.

Author contributions

All authors contributed equally.

Funding

Open Access funding enabled and organized by Projekt DEAL. This work was supported by the Federal Ministry of Education and Research of Germany and by the Sächsische Staatsministerium für Wissenschaft, Kultur und Tourismus in

³FAIR stands for Findable, Accessible, Interoperable and Reusable.

the programme Center of Excellence for AI-research “Center for Scalable Data Analytics and Artificial Intelligence Dresden/Leipzig”, project identification number: ScaDS.AI; and by the German Research Foundation (DFG) project NFDI4DataScience (no. 460234259). The funding bodies had no role in the design of the study, collection, analysis, and interpretation of data, or in writing the manuscript.

Data availability

No datasets were generated or analysed during the current study.

Declarations

Competing interests

The authors declare no competing interests.

Received: 7 July 2025 / Accepted: 27 February 2026

Published online: 12 March 2026

References

1. Clark C, Fraser K, Hand S, Hansen JG, Jul E, Limpach C, Pratt I, Warfield A (2005) Live migration of virtual machines. In Proceedings of the 2nd Conference on Symposium on Networked Systems Design & Implementation - Volume 2, USENIX Association.
2. Kaur J, Chana I (2018) Review of live virtual machine migration techniques in cloud computing. In 2018 International Conference on Circuits and Systems in Digital Enterprise Technology (ICCSDET), pp 1–6.
3. Imran M, Ibrahim M, Din MSU, Rehman MAU, Kim BS (2022) Live virtual machine migration: a survey, research challenges, and future directions. *Comput Electr Eng* 103:108297. <https://doi.org/10.1016/j.compeleceng.2022.108297>.
4. Lu P, Barbalace A, Palmieri R, Ravindran B (2014) Adaptive live migration to improve load balancing in virtual machine environment. In: Euro-par 2013: parallel processing workshops. Springer, Berlin, Heidelberg Berlin Heidelberg, pp 116–125.
5. Soma S, Rukmini S (2023) Virtual machine and container live migration algorithms for energy optimization of data Centre in cloud environment: a research review. In: Joby PP, Balas VE, Palanisamy R (eds) IoT based control networks and intelligent systems. Springer Nature Singapore, pp 637–647.
6. Sindi M, Williams JR (2019) Using container migration for HPC workloads resilience. In 2019 IEEE High Performance Extreme Computing Conference (HPEC), pp 1–10.
7. Garg R, Price G, Cooperman G (2019) MANA for MPI: MPI-agnostic network-agnostic transparent checkpointing. In Proceedings of the 28th international symposium on high-performance parallel and distributed computing, pp 49–60.
8. ISO 13008 (2022) Information and documentation — Digital records conversion and migration process. Standard, International Organization for Standardization.
9. Sarker TK, Tang M (2013) Performance-driven live migration of multiple virtual machines in datacenters. In 2013 IEEE International Conference on Granular Computing (GrC), pp 253–258.
10. Liu W, Fan T (2011) Live migration of virtual machine based on recovering system and CPU scheduling. In 2011 6th IEEE Joint International Information Technology and Artificial Intelligence Conference, vol 1. pp 303–307.
11. Rovnyagin MM, Sinelnikov DM, Varykhanov SS, Magazov TR, Kiamov AA, Shirokikh TA (2023) Intelligent Docker container orchestration for low scheduling latency and fast migration in paas. 2023 Seminar Inf Comput Process (ICP) 181–185.
12. Strunk A (2012) Costs of virtual machine live migration: a survey. In 2012 IEEE Eighth World Congress on Services, pp 323–329.
13. Shetty J, Anala MR, SG (2012) A survey on techniques of secure live migration of virtual machine. *Int J Comput Appl* 39(12):34–39. <https://doi.org/10.5120/4875-7305>.
14. Xu F, Liu F, Jin H, Vasilakos AV (2014) Managing performance overhead of virtual machines in cloud computing: a survey, state of the art, and future directions. *Proc. IEEE* 102(1):11–31. <https://doi.org/10.1109/JPROC.2013.2287711>.
15. Medina V, García JM (2014) A survey of migration mechanisms of virtual machines. *ACM Comput Surv* 46(3). <https://doi.org/10.1145/2492705>.
16. Li W, Kanso A (2015) Comparing containers versus virtual machines for achieving High availability. In 2015 IEEE International Conference on Cloud Engineering, pp 353–358.
17. Yamada H (2016) Survey on mechanisms for live virtual machine migration and its improvements. *Inf Media Technol* 11101–115. <https://doi.org/10.1118/5/mt.11.101>.
18. Zhang F, Liu G, Fu X, Yahyapour R (2018) A survey on virtual machine migration: challenges, techniques, and open issues. *IEEE Commun Surv Tutor* 20(2):1206–1243. <https://doi.org/10.1109/COMST.2018.2794881>.
19. Wang S, Xu J, Zhang N, Liu Y (2018) A survey on service migration in mobile Edge computing. *IEEE Access*. 6:23511–23528. <https://doi.org/10.1109/ACCESS.2018.2828102>.
20. Noshay M, Ibrahim M, Ali HA (2018) Optimization of live virtual machine migration in cloud computing: a survey and future directions. *J Retailing Network Comput Appl* 110:1–10. <https://doi.org/10.1016/j.jnca.2018.03.002>.
21. Rejiba Z, Masip-Bruin X, Marín-Tordera E (2019) A survey on mobility-induced service migration in the Fog, Edge, and related computing paradigms. *ACM Comput Surv* 52(5). <https://doi.org/10.1145/3326540>.
22. Le T (2020) A survey of live virtual machine migration techniques. *Comput Sci Rev* 38:100304. <https://doi.org/10.1016/j.cosrev.2020.100304>.
23. Masdari M, Khezri H (2020) Efficient VM migrations using forecasting techniques in cloud computing: a comprehensive review. *Clust Comput* 23(4):2629–2658. <https://doi.org/10.1007/s10586-019-03032-x>.
24. Ramanathan S, Kondepudi K, Razo M, Tacca M, Valcarengli L, Fumagalli A (2021) Live migration of virtual machine and container based mobile Core network components: a comprehensive study. *IEEE Access*. 9:105082–105100. <https://doi.org/10.1109/ACCESS.2021.3099370>.
25. Kaur K, Guillemin F, Sailhan F (2022) Container placement and migration strategies for cloud, fog, and edge data centers: a survey. *Int J Network Manage* 32(6):e2212. <https://doi.org/10.1002/nem.2212>.
26. Elsaid ME, Abbas HM, Meinel C (2022) Virtual machines pre-copy live migration cost modeling and prediction: a survey. *Distrib Parallel Databases* 40(2):441–474. <https://doi.org/10.1007/s10619-021-07387-2>.
27. He T, Buyya R (2023) A taxonomy of live migration management in cloud computing. *ACM Comput Surv* 56(3). <https://doi.org/10.1145/3615353>.
28. Soussi W, Gür G, Stiller B (2024) Democratizing container live migration for enhanced future networks - a survey. *ACM Comput Surv* 57(4):1–37. <https://doi.org/10.1145/3704436>.
29. Haris RM, Barhamgi M, Nhlabsati A, Khan KM (2024) Optimizing pre-copy live virtual machine migration in cloud computing using machine learning-based prediction model. *Computing* 106(9):3031–3062. <https://doi.org/10.1007/s00607-024-01318-6>.
30. Zhang L, Du H, Wang Y, Liu X (2024) Improved pre-copy container live migration optimization. In 2024 7th International Conference on Electronics Technology (ICET), pp 269–273.
31. Hines MR, Gopalan K (2009) Post-copy based live virtual machine migration using adaptive pre-paging and dynamic self-ballooning. In Proceedings of the 2009 ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments, Association for Computing Machinery, pp 51–60.
32. Puliafito C, Mingozzi E, Vallati C, Longo F, Merlino G (2018) Virtualization and migration at the network Edge: an overview. In 2018 IEEE International Conference on Smart Computing (SMARTCOMP), pp 368–374.
33. Nayak PC, Garg D, Shakva AK, Saini P (2018) A research paper of existing live VM migration and a hybrid VM migration approach in cloud computing. In 2018 2nd International Conference on Trends in Electronics and Informatics (ICOEI), pp 720–725.
34. Lu P, Barbalace A, Ravindran B (2013) HSG-LM: hybrid-copy speculative guest OS live migration without hypervisor. In Proceedings of the 6th International Systems and Storage Conference, pp 1–11.
35. Benjaponpitak T, Karakate M, Sripanidkulchai K (2020) Enabling live migration of containerized applications across clouds. In IEEE INFOCOM 2020 - IEEE Conference on Computer Communications, pp 2529–2538.
36. Jia J, Lin X, Lin F, Liu Y (2024) DCU-CHK: checkpointing for large-scale CPU-DCU heterogeneous computing systems. *CCF Trans High Perform Comput* 6(5):519–532. <https://doi.org/10.1007/s42514-023-00178-4>.
37. Gerofi B, Fujita H, Ishikawa Y (2010) An efficient process live migration mechanism for load balanced distributed virtual environments. In 2010 IEEE International Conference on Cluster Computing, pp 197–206.
38. Xu Y, Belyaev L, Jain T, Schafer D, Skjellum A, Cooperman G (2023) Implementation-oblivious transparent checkpoint-restart for MPI. In Proceedings of the SC’23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis, ACM, pp. 1738–1747.

39. Poggiani L, Puliafito C, Virdis A, Mingozzi E (2024) Live migration of multi-container Kubernetes pods in multi-Cluster serverless Edge systems. In Proceedings of the 1st Workshop on Serverless at the Edge, Association for Computing Machinery, pp. 9–16.
40. Azeem BA, Helal M (2023). Performance evaluation of checkpoint/restart techniques. arXiv preprint arXiv:2311.17545. doi: <https://doi.org/10.48550/arXiv.2311.17545>.
41. Garg R, Arya K, Cao J, Cooperman G, Evans J, Garg A, Rosenberg NA, Suresh K (2017). Adapting the DMTCP plugin model for checkpointing of hardware emulation. arXiv preprint arXiv:1703.00897. doi: <https://doi.org/10.48550/arXiv.1703.00897>.
42. Timalina M, Gerhardt L, Tyler N, Blaschke JP, Arndt W (2024). Optimizing checkpoint-restart mechanisms for HPC with DMTCP in containers at NERSC. arXiv preprint arXiv:2407.19117.
43. Xu Y, Cooperman G (2024) Enabling practical transparent checkpointing for MPI: a topological sort approach. In 2024 IEEE International Conference on Cluster Computing (CLUSTER), IEEE, pp. 238–249.
44. Pickartz S, Eiling N, Lankes S, Razik L, Monti A (2016) Migrating Linux containers using CRIU. In: Tauber M, Mohr B, Kunkel JM (eds) High performance computing. Springer International Publishing, pp 674–684.
45. Singh R, Arun K, Mishra D (2022) LDT: lightweight dirty tracking of memory pages for x86 systems. In 2022 IEEE 29th International Conference on High Performance Computing, Data, and Analytics (HiPC), IEEE, pp. 85–94.
46. Eskandani N, Koziolk H, Hark R, Linsbauer S (2024) The state of container checkpointing with CRIU: a multi-case experience report. In 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C), pp 54–59.
47. Tošić A (2024) Run-time application migration using Checkpoint/Restore in Userspace. *J Web Eng* 23(5):735–748. <https://doi.org/10.13052/jwe1540-9589.2357>.
48. Ma L, Yi S, Li Q (2017) Efficient service handoff across edge servers via docker container migration. In Proceedings of the Second ACM/IEEE Symposium on Edge Computing, Association for Computing Machinery.
49. Qiu Y, Lung CH, Ajila S, Srivastava P (2017) LXC container migration in cloudlets under multipath TCP. In 2017 IEEE 41st Annual Computer Software and Applications Conference (COMPSAC), vol 2. pp 31–36.
50. Xu B, Wu S, Xiao J, Jin H, Zhang Y, Shi G, Lin T, Rao J, Yi L, Jiang J (2020) Sledge: towards efficient live migration of Docker containers. In 2020 IEEE 13th International Conference on Cloud Computing (CLOUD), pp 321–328.
51. Bautista-Gomez L, Tsuboi S, Komatitsch D, Cappello F, Maruyama N, Matsuoka S (2011) FTI: high performance fault tolerance Interface for hybrid systems. In SC'11: Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis, pp 1–12.
52. MTEC (2025) https://docs.redhat.com/en/documentation/openshift_container_platform/4.5/html/migration_toolkit_for_containers/index. Accessed: 11 Feb 2025.
53. Gamess E, Parajuli M (2024) Containers unleashed: a raspberry pi showdown between Docker, Podman, and LXC/LXD. *SoutheastCon 2024* 34–43.
54. Govindaraj K, Artemenko A (2018) Container live migration for latency critical industrial applications on Edge computing. In 2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA), vol 1. pp 83–90.
55. Silva D, Rafael J, Fonte A (2024) Toward optimal virtualization: an updated comparative analysis of Docker and LXD container technologies. *Computers* 13(4). <https://doi.org/10.3390/computers13040094>.
56. L-I (2025) <https://linuxcontainers.org/lxc/introduction>. Accessed: 15 Jan 2025.
57. Das R, Sidhanta S (2023) Live migration of containers in the Edge. *SN Comput Sci* 4(5):479. <https://doi.org/10.1007/s42979-023-01871-5>.
58. Vazquez A (2024) Linux containers (LXC). Apress, pp 363–433.
59. Harter T, Salmon B, Liu R, Arpacı-Dusseau AC, Arpacı-Dusseau RH (2016) Slacker: fast distribution with lazy Docker containers. In 14th USENIX Conference on File and Storage Technologies (FAST 16), USENIX Association, pp. 181–195.
60. Nadgowda S, Suneja S, Bila N, Isci C (2017) Voyager: complete container state migration. In 2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS), pp 2137–2142.
61. Cao J, Yu Z, Xue B (2024) Research on collaborative edge network service migration strategy based on crowd clustering. *Sci Rep* 14. <https://doi.org/10.1038/s41598-024-58048-0>.
62. Nath SB, Addya SK, Chakraborty S, Ghosh SK (2025) CSMD: container state management for deployment in cloud data centers. *Futu Gene Com Sys* 162:107495. <https://doi.org/10.1016/j.future.2024.107495>.
63. Wang K, Xie Z (2023) Optimization of docker container live migration solution. In: Yan S, Loskot P (eds) Eighth international symposium on advances in electrical, electronics, and computer engineering (ISAEECE 2023), vol 12704. International Society for Optics and SPIE, Photonics, p 1270401.
64. P-G (2025) <https://github.com/containers/podman>. Accessed: 8 Jan 2025.
65. P-C (2025) <https://podman.io/docs/checkpoint>. Accessed: 8 Jan 2025.
66. Reber A (2019) Container live migration. Available at: <https://lisas.de/adrian/container-live-migration-article.pdf>.
67. A (2025) <https://scc.dipc.org/docs/>. Accessed: 6 Mar 2025.
68. Andrijauskas F, Sfiligoi I, Davila D, Arora A, Guiang J, Bockelman B, Thain G, Würthwein F (2024) CRIU-Checkpoint Restore in userspace for computational simulations and scientific applications. In EPJ Web of Conferences, vol 295. EDP Sciences, pp. 07046.
69. DSP (2025) <https://github.com/lancium/dmtcp-singularity-plugin>. Accessed: 15 Jan 2025.
70. MSC (2025) <https://github.com/mmores500/mwe-singularity-checkpoint>. Accessed: 15 Jan 2025.
71. C-F (2026) <https://apptainer.org/docs/user/main/checkpoint.html>. Accessed: 16 Jan 2026.
72. Patrão L (2024) VMware and vSphere overview. Apress, pp 9–18.
73. Elsaid MEM (2022) Virtual machines live migration cost modeling and prediction. Ph. D. thesis, Universität Potsdam.
74. Elsaid ME, Abbas HM, Meinel C (2020) Live migration timing optimization for VMware environments using machine learning techniques. In Proceedings of the 10th International Conference on Cloud Computing and Services Science - CLOSER, SciTePress, INSTICC, 91–102.
75. VLMO (2025) <https://learn.microsoft.com/>. Accessed: 13 Feb 2025.
76. Kapil D, Mishra SK, Gupta VK (2022) A performance perspective of live migration of virtual machine in cloud data center with future directions. *Int J Multiling Wireless Microwave Technol* 12(4):48–56. <https://doi.org/10.5815/ijwmt.2022.04.04>.
77. XPBG (2025) https://wiki.xenproject.org/wiki/Xen_Project_Beginners_Guide. Accessed: 13 Feb 2025.
78. Storniole F, Leonardi L, Lettieri G (2024) Improving live migration efficiency in QEMU: an eBPF-based paravirtualized approach. *J Syst Archit* 150:103130. <https://doi.org/10.1016/j.sysarc.2024.103130>.
79. Li JL, Li SW (2025) Performance implications of SEV virtual machine live migration. In: Euro-par 2024: parallel processing workshops. Springer Nature Switzerland, Cham, pp 128–139.
80. Yang FJ, Li JL, Xue K, Li SW (2024) Designing and implementing live migration support for Arm-based confidential VMs. In Proceedings of the 15th ACM SIGOPS Asia-Pacific Workshop on Systems, Association for Computing Machinery, New York, NY, USA, 53–59.
81. Q-M (2025) <https://www.qemu.org/docs/master/devel/migration/index.html>. Accessed: 2 Jul 2025.
82. Mirkhanzadeh B, Zhang T, Razo-Razo M, Tacca M, Fumagalli A (2021) Super-Cloudlet: rethinking Edge computing in the Era of open optical networks. In 2021 International Conference on Computer Communications and Networks (ICCCN), pp 1–11.
83. CLM (2025) <https://docs.openstack.org/nova/latest/admin/configuring-migrations.html>. Accessed: 11 Jan 2025.
84. Walters JP, Chaudhary V, Cha M, Guercio S, Gallo S (2008) A comparison of virtualization technologies for HPC. In 22nd International Conference on Advanced Information Networking and Applications (aina 2008), pp 861–868.
85. Di Z, Shao E, He M (2020) Reducing the time of live container migration in a workflow. In IFIP International Conference on Network and Parallel Computing, Springer, pp. 263–275.
86. CALM (2026) https://wiki.openvz.org/Checkpointing_and_live_migration. Accessed: 16 Jan 2026.
87. Indukuri PSV (2016) Performance comparison of Linux containers (LXC) and OpenVZ during live migration: an experiment. Master's thesis, Blekinge Institute of Technology.
88. Habib I (2008) Virtualization with KVM. *Linux J* 2008(166):8.
89. Karlsson E (2024) Kubernetes as a virtual machine management system : a comparative study of KubeVirt and traditional approaches. Master's thesis, KTH, School of Electrical Engineering and Computer Science (ECS).
90. Kampadai K, Chazapis A, Bilas A (2025). Optimizing the Longhorn cloud-native software defined storage Engine for High performance. arXiv preprint arXiv:2502.14419. doi: <https://doi.org/10.48550/arXiv.2502.14419>.
91. H (2025) <https://docs.harvesterhci.io/v1.4/vm/live-migration>. Accessed: 11 Jan 2025.

92. H-S (2026) <https://learn.microsoft.com/en-us/azure/security/fundamentals/hypervisor> Accessed: 20 Jan 2026.
93. Barbalace A, Karaoui ML, Wang W, Xing T, Olivier P, Ravindran B (2020) Edge computing: the case for heterogeneous-ISA container migration. In Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments, Association for Computing Machinery, pp. 73–87.
94. Bapat A, Shastri J, Wang X, Sundarasamy A, Ravindran B (2024) Dapper: a lightweight and extensible framework for live program state rewriting. In 2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS), pp 738–749.
95. Wruck F, Peisl M, Epple C, Weiß M (2024) HETCOM: heterogeneous container migration based on TEE- or TPM-established Trust. In Proceedings of the 2024 ACM Workshop on Secure and Trustworthy Cyber-Physical Systems, Association for Computing Machinery, pp. 51–60.
96. Yang Y, Hu A, Zheng Y, Zhao B, Zhang X, Quinn A (2024). Transparent and efficient live migration across heterogeneous hosts with Wharf. arXiv preprint arXiv:2410.15894. doi: <https://doi.org/10.48550/arXiv.2410.15894>.
97. Chen S, Zhou M (2021) Evolving container to unikernel for Edge computing and applications in process industry. *Processes* 9(2). <https://doi.org/10.3390/pr9020351>.
98. Cozzolino V, Flum O, Ding AY, Ott J (2020) MirageManager: enabling stateful migration for unikernels. In Proceedings of the Workshop on Cloud Continuum Services for Smart IoT Systems, CCloudT'20, Association for Computing Machinery, New York, NY, USA, 13–19.
99. Chen Z, Zhan Y, Hu P, Zhao X, Yang M, Tan S, Zhang L, Lu L, Yin J, Chen Z (2025) UKFaaS: lightweight, High-performance and secure FaaS communication with unikernel. *IEEE Trans Intell Transp Syst Comput* 74(10):3305–3318. <https://doi.org/10.1109/TC.2025.3586031>.
100. Mansouri Y, Babar MA (2021) A review of edge computing: features and resource virtualization. *J Parallel Distrib Comput* 150:155–183. <https://doi.org/10.1016/j.jpdc.2020.12.015>.
101. C-K (2026) <https://kubernetes.io/blog/2023/07/06/confidential-kubernetes/>. Accessed: 2 Jan 2026.
102. Kollenda K (2023). General overview of AMD SEV-SNP and Intel TDX.
103. Intel-TDMig (2026) <https://www.intel.com/content/www/us/en/content-details/839200/intel-tdx-module-partitioning-architecture-specification-what-changed.html>. Accessed: 2 Jan 2026.
104. Choudhary A, Govil M, Singh G et al. (2017) A critical survey of live virtual machine migration techniques. *J Cloud Comp* 6:23. <https://doi.org/10.1186/s13677-017-0092-1>.
105. Haris RM, Khan KM, Nhlabatsi A (2022) Live migration of virtual machine memory content in networked systems. *Comput Networks* 209:108898. <https://doi.org/10.1016/j.comnet.2022.108898>.
106. Hosseini Shirvani M, Rahmani AM, Sahafi A (2020) A survey study on virtual machine migration and server consolidation techniques in DVFS-enabled cloud datacenter: taxonomy and challenges. *J Educ Chang King Saud Univ - Comput Inf Sci* 32(3):267–286. <https://doi.org/10.1016/j.jksuci.2018.07.001>.
107. Widjajarto A, Jacob DW, Lubis M (2021) Live migration using checkpoint and restore in userspace (CRIU): usage analysis of network, memory and CPU. *Bull Electr Eng Inf* 10(2):837–847. <https://doi.org/10.11591/eei.v10i2.2742>.
108. Russel EFJ (2022) Rapture: an efficient cloud gaming platform built on containerization. Master thesis, Delft University of Technology.
109. Xu X, Zhang N, Cui M, He M, Surana R (2019) Characterization and prediction of performance interference on mediated passthrough GPUs for interference-aware scheduler. In 11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19).
110. Tian K, Dong Y, Cowperthwaite D (2014) A full GPU virtualization solution with mediated pass-through. In 2014 USENIX Annual Technical Conference (USENIX ATC 14), pp 121–132.
111. Ma J, Zheng X, Dong Y, Li W, Qi Z, He B, Guan H (2018 March) gMig: Efficient GPU Live Migr Optimized Softw Dirty Page Full Virtualiz. *SIGPLAN Not* 53(3):31–44. <https://doi.org/10.1145/3296975.3186414>.
112. Belkhiari A, Dagenais M (2024) Analyzing GPU performance in virtualized environments: a case study. *Future Internet* 16(3). <https://doi.org/10.3390/fi16030072>.
113. Huang Z, Wei X, Hao Y, Chen R, Han M, Gu J, Chen H (2024). PARALLELGPUOS: a concurrent OS-level GPU checkpoint and Restore system using validated speculation. arXiv preprint arXiv:2405.12079. doi: <https://doi.org/10.48550/arXiv.2405.12079>.
114. Han M, Shen W, Peng G, Chen R, Chen H (2024). Microsecond-scale dynamic validation of idempotency for GPU kernels. arXiv preprint arXiv:2410.23661. doi: <https://doi.org/10.48550/arXiv.2410.23661>.
115. Nukada A, Suzuki T, Matsuoka S (2023) Efficient checkpoint/Restart of CUDA applications. *Parallel Comput* 116(C). <https://doi.org/10.1016/j.parco.2023.103018>.
116. Stoyanov R, Spišáková V, Ramos J, Gurfinkel S, Vagin A, Reber A, Armour W, Bruno R (2025) CRIUgpu: Transparent Checkpointing of GPU-Accelerated Workloads.
117. Supporting-ROCm-with-CRIU (2025) <https://github.com/checkpoint-restore/criu/blob/criu-dev/plugins/amdgpu/README.md>. Accessed: 18 Feb 2025.
118. Polyakov AY, Shalom G, Schwartz A, Yehezkel A, David OB, Kahalon O, Shahar A, Liss L (2025) Device-assisted live migration of RDMA devices. In Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP '25, Association for Computing Machinery, New York, NY, USA, 153–168.
119. Li X, Shu R, Xiong Y, Ren F (2024) Software-based live migration for containerized RDMA. In Proceedings of the 8th Asia-Pacific Workshop on Networking, pp 52–58.
120. Planeta M, Bierbaum J, Antony LSD, Hoefer T, Härtig H (2021) MigrOS: transparent live-migration support for containerised RDMA applications. In 2021 USENIX Annual Technical Conference (USENIX ATC 21), USENIX Association, pp. 47–63.
121. Attar-Khorasani S, Azmi R, Sabeti V (2017) SDSuPK: secured data sharing using proxy kerberos to improve openstack swift security. In 2017 7th International Conference on Computer and Knowledge Engineering (ICCKE), pp 77–83.
122. Munhoz V, Castro M (2024) Enabling the execution of HPC applications on public clouds with HPC@ cloud toolkit. *Concurrency Comput Pract Exper* 36(8):e7976. <https://doi.org/10.1002/cpe.7976>.
123. Sherpa L, Attar-Khorasani S, Sankar R, Müller-Pfefferkorn R, Ghiasvand S (2024) Keensight: cloud based collaborative environment for streamlining machine learning development. In 2024 15th International Conference on Information, Intelligence, Systems and Applications (IISA), pp 1–8.
124. Breitgand D, Kutiel G, Raz D (2010) Cost-aware live migration of services in the cloud. In Proceedings of the 3rd Annual Haifa Experimental Systems Conference, Association for Computing Machinery.
125. Ye K, Jiang X, Huang D, Chen J, Wang B (2011) Live migration of multiple virtual machines with resource reservation in cloud computing environments. In 2011 IEEE 4th International Conference on Cloud Computing, pp 267–274.
126. Gupta A, Namasudra S (2022) A novel technique for accelerating live migration in cloud computing. *Autom Softw Eng* 29:34. <https://doi.org/10.1007/s10515-022-00332-2>.
127. Gao Q, Li X, Wang X, Song L, Ding Y, Liu Y, Sun Y (2024) TrustMig: a container migration architecture model based on TEE. In 2024 IEEE 7th International Conference on Computer and Communication Engineering Technology (CCET), pp 280–285.
128. Gao Q, Song L, Lei Y, Wang F, Wang L, Zong S, Ding Y (2024) Enhancing consistency in container migration via TEE: a secure architecture. In 2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), pp 21–28.
129. Wang J, Mahmoody P, Brasser F, Jauernig P, Sadeghi AR, Yu D, Pan D, Zhang Y (2022) VirTEE: a full backward-compatible TEE with native live migration and secure I/O. In Proceedings of the 59th ACM/IEEE Design Automation Conference, DAC '22, Association for Computing Machinery, New York, NY, USA, 241–246.
130. Vasileiadis S, Giannetos T, Schunter M, Crispo B (2025). Reinforcing secure live migration through verifiable state management.
131. Kaur K, Guillemin F, Sailhan F (2023) Live migration of containerized microservices between remote Kubernetes clusters. In INFOCOM WKSHPS 2023: IEEE International Conference on Computer Communications - Workshop, pp 114–119.
132. Vasireddy I, Wankar R, Chillarige RR (2024) Pod migration with optimized containers using persistent volumes in Kubernetes. In Iglesias A, Shin J, Patel B, Joshi A (eds) Proceedings of World Conference on Information Systems for Business Management, Springer Nature Singapore, pp. 27–36.
133. Sahni Y, Cao J, Yang L (2019) Data-aware task allocation for achieving low latency in collaborative Edge computing. *IEEE Internet Things J* 6(2):3512–3524. <https://doi.org/10.1109/JIOT.2018.2886757>.
134. Khan WZ, Ahmed E, Hakak S, Yaqoob I, Ahmed A (2019) Edge computing: a survey. *Futu Gene Com Sys* 97:219–235. <https://doi.org/10.1016/j.future.2019.02.050>.
135. Fujii D, Matsubara K, Nakata Y (2024) Stateful VM migration among heterogeneous WebAssembly runtimes for efficient Edge-cloud collaborations. In Proceedings of the 7th International Workshop on Edge Systems, Analytics and Networking, EdgeSys '24, Association for Computing Machinery, New York, NY, USA, 19–24.

136. Tinto E, Marchiori L, Vardanega T (2025) Live migration of compiled Wasm modules across the compute continuum. *J Syst Archit* 168:103532. <https://doi.org/10.1016/j.sysarc.2025.103532>.
137. Attar-Khorasani S, Chalmers R (2021) Internet of things data visualization for business Intelligence. *Big Data* 12(6):478–503. <https://doi.org/10.1089/big.2021.0200>.
138. Alqam S, Alzeidi N, Touzene A, Day K (2023) Reinforcement learning-driven decision-making for live virtual machine migration in Fog computing. *J Eng Res [TJER]* 20(2):113–122. <https://doi.org/10.53540/tjer.vol20iss2pp113-122>.
139. Alqam SS, Zeidi NA, Touzene A, Day K (2023) Location-aware reinforcement learning-based live virtual machine migration in Fog computing. In 2023 16th International Conference on Advanced Computer Theory and Engineering (ICACTE), IEEE, pp. 242–248.
140. Osanaiye OA, Chen S, Yan Z, Lu R, Choo KR, Dlodlo ME (2017) From cloud to Fog computing: a review and a conceptual live VM migration framework. *IEEE Access*. 5:8284–8300. <https://doi.org/10.1109/ACCESS.2017.2692960>.
141. Talebian A, Valera A, Sahni J, Seah WK (2022) Robust intra-slice migration in Fog computing. In 2022 IEEE 47th Conference on Local Computer Networks (LCN), IEEE, pp. 48–55.
142. Sherpa L, Khaydarov V, Müller-Pfefferkorn R (2024 Dec) Fairness along the machine learning lifecycle using dataverse in combination with MLflow. *Data Sci J* 23(1). <https://doi.org/10.5334/dsj-2024-055>.
143. ISO 25011 (2017) Information technology — systems and software quality requirements and evaluation (SQuaRE) — service quality models. Standard, International Organization for Standardization.
144. ISO 25010 (2022) Systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — product quality model. Standard. International Organization for Standardization.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.